

Практикалық жұмыс №2

ІЗДЕУ АЛГОРИТМДЕРІ МЕН ЭВРИСТИКАЛАР

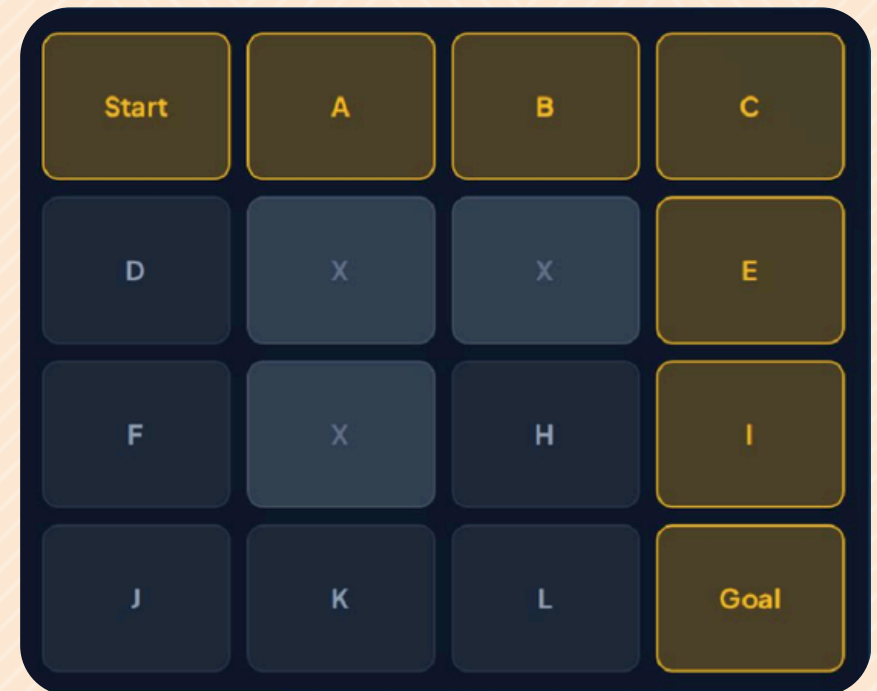
DFS, BFS, UCS және A* Алгоритмдерін Лабиринтте қолдану

1. DFS Алгоритмі (Depth-First Search)

Принципі: Тереңдікке қарай қозғалады. LIFO Стек (Stack) қолданады.

```
def dfs(graph, start, goal):
    stack = [(start, [start])]
    visited = set()
    while stack:
        node, path = stack.pop()
        if node == goal: return path
        if node not in visited:
            visited.add(node)
            for neighbour in reversed(graph.get(node, [])):
                stack.append((neighbour, path + [neighbour]))
```

Нәтиже: Start -> A -> B -> C -> E -> I -> Goal



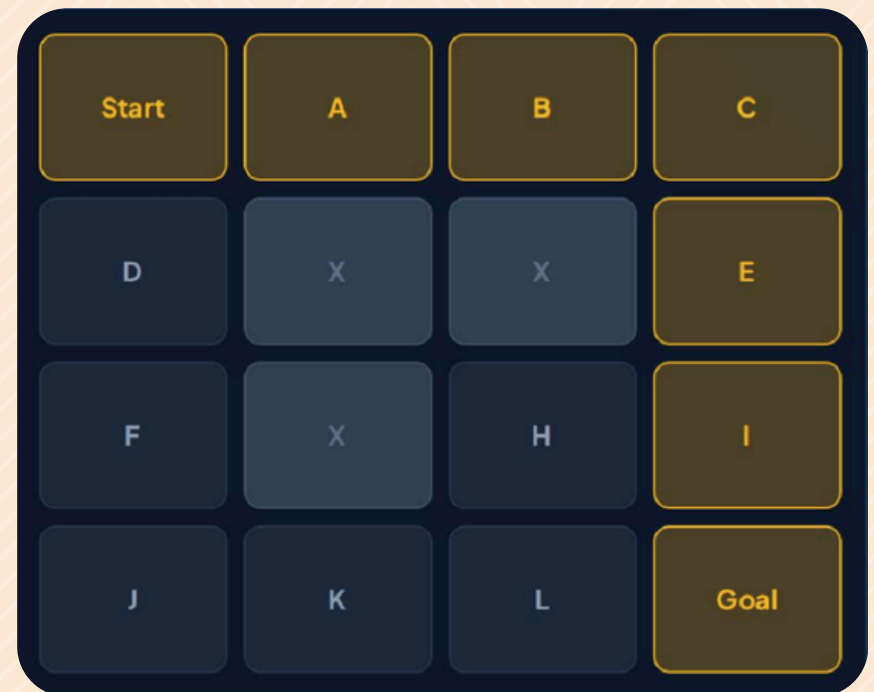
2. BFS Алгоритмі (Breadth-First Search)

Принципі: Ендікке қарай деңгейлеп іздейді. FIFO Кезек (Queue) қолданады.

```
from collections import deque

def bfs(graph, start, goal):
    queue = deque([(start, [start])])
    visited = {start}
    while queue:
        node, path = queue.popleft()
        if node == goal:
            return path
        for neighbour in graph.get(node, []):
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append((neighbour, path + [neighbour]))
```

Нәтиже: Start -> A -> B -> C -> E -> I -> Goal



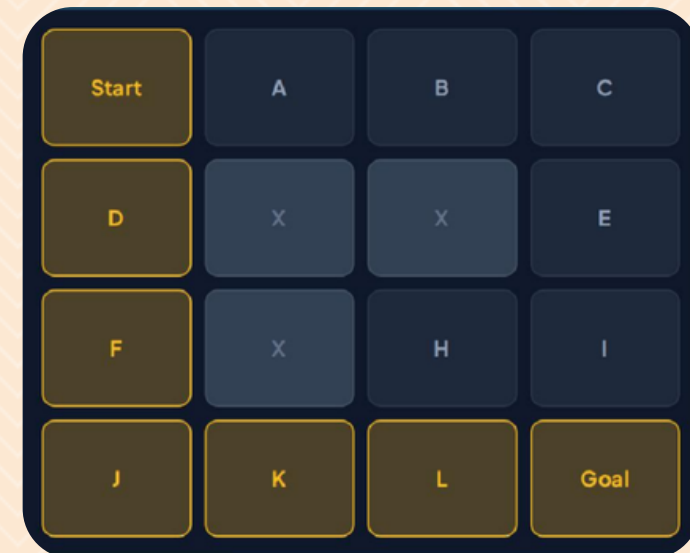
3. UCS Алгоритмі (Uniform Cost Search)

Принципі: Бастапқы түйіннен жиналған $g(n)$ салмағы ең аз жолды таңдайды (Priority Queue).

```
import heapq

defucs(graph,start, goal):
    frontier=[(0, start, [start])]
    visited = set()
    while frontier:
        cost,node, path = heapq.heappop(frontier)
        ifnode== goal: return path, cost
        ifnodenot in visited:
            visited.add(node)
            forneighbour, step_cost in graph.get(node, []):
                heapq.heappush(frontier, (cost + step_cost, neighbour, path + [neighbour]))
```

Нәтиже: Start -> D -> F -> J -> K -> L -> Goal | **Cost:** 9

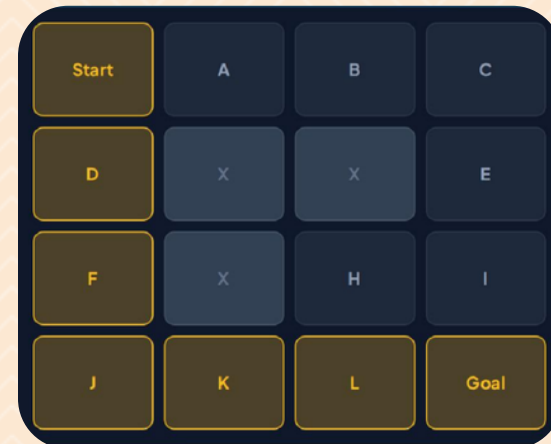


4. A* Search Алгоритмі

Принципі: $f(n) = g(n) + h(n)$ есептейді. Өтілген салмақ пен болжамды қашықтықты қатар ескереді.

```
import heapq

def astar(graph, start, goal, h):
    frontier=[(h[start],0,start, [start])]
    visited = set()
    while frontier:
        f, g, node, path = heapq.heappop(frontier)
        if node==goal: return path, g
        if node not in visited:
            visited.add(node)
            for neighbour, step_cost in graph.get(node, []):
                new_g = g + step_cost
                heapq.heappush(frontier, (new_g + h[neighbour], new_g, neighbour, path + [neighbour]))
```



Нәтиже: Start -> D -> F -> J -> K -> L -> Goal | **Cost:** 9

Теориялық сұрақтар

- **1. Күй (State) деген не?**

Лабиринттегі күй — бұл робот орналасқан нақты координаттар немесе түйін атауы

- **2. Шекарада (Frontier) не сақталады?**

Шекарада көрінген, бірақ әлі зерттелмеген түйіндер, оларға жеткен жолдар және басымдық мәндері сақталады

- **3. DFS vs BFS құрылымы**

Жауап: DFS **LIFO (Стек)** арқылы графтың ең тереңіне бұрын өтеді. BFS **FIFO (Кезек)** арқылы графты деңгейлеп, ең таяз (қадамы аз) маршрутты табу үшін қолданады.

- **4. Маршрут ұзындығы мен құнының айырмашылығы неде?**

Ұзындық (Length): басып өтілген қадамдар/қабырғалар саны. Құн (Cost): өткен қабырғалардың салмақтар жиынтығы.

Теориялық сұрақтар

- **5. Неліктен BFS салмақты графта ең арзан емес маршрутты қайтаруы мүмкін?**

BFS салмақты ескермейді, тек қадам санын азайтады, сондықтан ол қымбат жолды таңдауы мүмкін.

- **6. Неге UCS $g(n)$ қолданады?**

UCS бастапқы түйіннен жиналған $g(n)$ салмағын бақылап, Priority Queue арқылы әрқашан ең арзан маршрутты кеңейтеді.

- **7. $h(n)$ қандай ақпарат береді?**

Эвристика $h(n)$ — ағымдағы түйіннен мақсатқа дейінгі қалған арақашықтықтың болжамды бағасын береді.

- **8. Неге A^* екеуін де - $g(n)$ мен $h(n)$ қолданады?**

$g(n)$ басынан өткен нақты шығынды бақыласа, $h(n)$ мақсатқа қарай бағыттайды.
 $f(n)=g(n)+h(n)$ оңтайлы әрі тез іздеуді қамтамасыз етеді.

Теориялық сұрақтар

- **9. Эвристиканың жол берілетін (admissible) болуы нені білдіреді?**

Эвристика мақсатқа дейінгі нақты құнды ешқашан асыра бағаламайды. Бұл A^* -дың оңтайлығын қамтамасыз етеді.

- **10. Эвристиканың жол берілетін (admissible) болуы нені білдіреді?**

Егер $h(n) = 0$ болса, $f(n) = g(n) + 0 = g(n)$ болып, A^* алгоритмінің жұмысы толығымен UCS алгоритміне айналады.

- **11-Сұрақ: Visited құрылымдарын сақтау**

Жауап: Графтағы циклдерге (айналып қалуға) ұшырамау и бұрын өңделген түйіндерді қайталамау арқылы ресурсты үнемдеу үшін сақтаймыз.

- **12-Сұрақ: Қай алгоритм ең тиімді болды?**

Жауап: **A^* алгоритмі** ең тиімді болды. Ол минималды салмағы бар жолды ($Cost = 9$) тапты және эвристиканың $h(n)$ арқасында UCS-ке қарағанда азырақ түйіндерді кеңейтті.

Қорытынды және Дереккөздер

Қорытынды:

1. Салмақсыз графтарда қадам санын азайту үшін **BFS** тиімді.
2. **DFS** тереңдікке жылдам кетеді,
2. Салмақталған графтарда ең арзан жолды табу үшін **UCS** оңтайлы.
3. Эвристикалық ақпарат бар жағдайда ең аз ресурс жұмсап, арзан жолды табатын ең үздік алгоритм — **A***.

Дереккөздер (Sources):

- [geeksforgeeks](#) - мысалдар, анықтамалар
- Практикалық жұмыс материалдары — Week 2

colab: [сілтеме](#)

github: [сілтеме](#)